

Week 3 Tutorial

Welcome to Week 3 Tutorial. Please follow these instructions step by step - your tutors will take you through these one by one in class. Please ask for their assistance if you're stuck at any place - we are here to help.

Today, you will learn the following:

- Lecture 3 review
- Creating Objects
- Functions: `makeRGBSimple()`
- Functions: `makeRGB()`
- Functions: `drawRect()`

The code for `makeRGB()` and `drawRect()` is also separately provided as `Week3.html` and `script.js`. But please ensure that you type in and develop the code yourself - simply copying and pasting, or running the files we have provided will not be enough to ensure that you maximise your learning. To learn and retain the concepts, please do the whole exercise yourself, preferably by not using the files we have provided.

1: Lecture 3 Review

Your tutors will walk you through all of the sections and steps of Lecture 3, so that you learn the basics of how to define and call functions.

2. Objects - defining and accessing properties

In this exercise, we introduce the concept of objects using the example of a 'person' object. This object contains the properties `firstName`, `lastName`, `age`, and `eyeColour`.

Objects are great at storing related data in a structured way that is easy to access programatically.

In the following example we use the [object initializer](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Object_initializer)  (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Object_initializer) syntax to create the object.

To access the properties of the object, we can either use dot notation or bracket notation.

```
// Create an object using name: value pairs
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
  eyeColour: "blue"
```

```
};

// When we want to access properties from the object, we can use the dot notation
console.log(person.firstName);

// or we can use the bracket notation
console.log(person["firstName"]);

// We can access all the different values in the object
console.log(person.firstName + " " + person.lastName + " is " + person.age + " years old.");
```

3: Functions: makeRGBSimple()

Objective: Write a simple function that can produce the string we need to style HTML objects with an RGB color.

Last week we learned to use the backtick notation to create a string that was correctly formatted and took in numeric values. As we will use this formatting a lot, we will make a function that takes in the values and outputs the string we need.

- The function will require three parameters (arguments) red, green, and blue, each of these arguments stands for a numerical value for three numbers.
- The function will then output a string that includes the formatted input values, ready to be used in a `setAttribute()` function.

Here it is - it is really quite simple:

Take a careful look at the function definition, the red, green and blue values are place holders in the function definition, they do not really exist, but the function definition shows what would happen to each of them when passed into the function when it is called.

```
// A CSS colour string is formed based on the provided red, green and blue input values.
function formatRGB(red, green, blue) {
    return `rgb(${red}, ${green}, ${blue})`;
}
```

Now let's call the function. In this case the function takes in three arguments, for the three parameters, red, green, and blue, and spits out or returns a string formatted as `rgb(100, 100, 200)`.

```
// Now we can call this function with three arguments for the three parameters.
formatRGB(100, 100, 200); // returns 'rgb(100, 100, 200)'
```

Note the format for the return output - you are returning a **string**, but you want to use the input that the user passes through the **variable names or numeric values** red, green, and blue, so you use the format `${variable_name}`, which signifies that the **string representation of the value of the variable (or number)** will be part of the output string.

4: Functions: makeRGB ()

Objective: Make this function more flexible by letting the user choose between random values and user specified values.

Challenges: Allow for null values to be passed in as arguments.

- Check if any of the values is null.
 - If any of the input values is null, generate a random number and then pass that new value to the string.
 - If any of the input values is not null, pass that value to the string as before.

Desired results: The new function will allow for new kinds of input, so we can define an RGB string by:

- Specifying three numbers, e.g. (199, 100, 255).
- Specifying nothing, and completely randomising colour choice.
- Specifying one of the colour values, and leaving the others random, e.g. (null, null, 100) (this could let you generate random colours that all have a medium blue tint, because the blue value is fixed at 100)
- Specifying two of the colour values, and leaving the other random, e.g. (null, 100, 100).

Solutions:

We will use a specific notation that lets us make choices depending on the value of something. Here we will use ?? known as the nullish coalescing operator.

This works in a simple way:

- It checks if the value to the left of the ?? is null.
 - If it is null it swaps the value on the left for the value on the right (this can also be a function that returns a value)
 - If it is not null, it returns the value as is.

Implementation:

The new function takes in three parameters, red, green, and blue. We then use the ?? to check if the user has provided a specific number as input, or has provided a null or undefined input.

If the null condition tests true, then we generate a random number, otherwise we pass the actual value.

Finally, we pass these into a string, which is the output of the function. Let's have a look at the function definition:

```
// The function takes in three parameters, red, green, and blue.
// We use a logical test to check if the user has provided
// a specific number as input, or has provided a null or
// undefined input.

// If the null condition tests true, then we generate a random number.
// Finally, we pass these into a string, which is the output of
// the function.

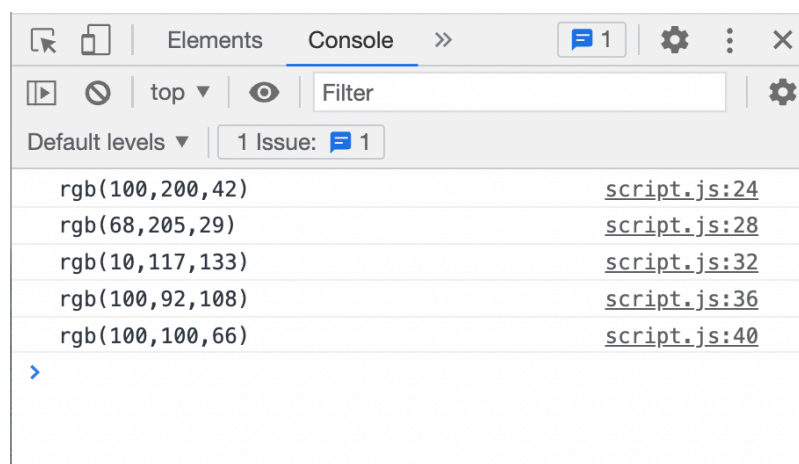
function makeRGB(redInputValue, greenInputValue, blueInputValue) {
  // Here, we test if there is a specific user input. If yes, then that value is assigned
  // to the output. Otherwise, a random value is generated.
  let redOutputValue = redInputValue ?? Math.round(Math.random() * 255);
  let greenOutputValue = greenInputValue ?? Math.round(Math.random() * 255);
  let blueOutputValue = blueInputValue ?? Math.round(Math.random() * 255);

  // Now we define a string and pass these output values into a string.
  return `rgb(${redOutputValue},${greenOutputValue},${blueOutputValue})`;
}
```

Now we have defined the function we can use it and check the output (note how the null values are always replaced with random values)

```
// Now we can test our function using different combinations
// of inputs.
// All three numbers given by user.
let definedColour = makeRGB(100, 200, 42);
console.log(definedColour);
// None of the numbers given by user, totally random.
let randomColour = makeRGB(null, null, null);
console.log(randomColour);
// Note that this will work too.
let randomColour1 = makeRGB();
console.log(randomColour1);
// One of the dimensions specified, the other two null.
let partiallyDefinedColour = makeRGB(100, null, null);
console.log(partiallyDefinedColour);
// Two of the dimensions specified, the other one null.
let partiallyDefinedColour1 = makeRGB(100, 100, null);
console.log(partiallyDefinedColour1);
```

This runs as follows:



Let's make this even more modular and flexible by creating another function to take care of the random number scaling.

```
// But wait! we just learnt about how not to repeat ourselves with functions. Let's add a func
tion to make the make RGB function less noisy.
```

```
// Generate a random integer value between 0 and the provided magnitude (limit)
function randomInt(magnitude) {
  let integer = Math.random();
  integer *= magnitude;
  integer = Math.floor(integer);

  return integer;
}

function makeRGB(redInputValue, greenInputValue, blueInputValue) {
  // Here, we test if there is a specific user input. If yes, then that value is assigned
  // to the output. Otherwise, a random value is generated.
  let redOutputValue = redInputValue ?? randomInt(255);
  let greenOutputValue = greenInputValue ?? randomInt(255);
  let blueOutputValue = blueInputValue ?? randomInt(255);

  // Now we define a string and pass these output values into a string.
  return `rgb(${redOutputValue},${greenOutputValue},${blueOutputValue})`;
}

// That's better!
```

5: Functions: drawRect()

Now that we have a way to generate an RGB string in an arbitrary way, let's write a function to draw a rectangle. The inputs to this function are parameters like x-position, y-position, width and height, and colour. The output will be a rectangle drawn onto the SVG canvas each time the function is called.

Objective: Make a function that draws a rectangle with specified OR random colours and positions.

Challenges: Allow for null values to be passed in as arguments for positions, and re-use the RGB function we already defined.

Make sure the positions and size of the rectangle are constrained so that it fits onto the SVG we draw it in.

Desired results: The new function will create a new rectangle either partially or completely defined by function arguments or randomly

- Specifying seven numbers, e.g. (199, 100, 255, 100, 100, 150, 200)
- Specifying nothing, and completely randomising colour choice
- Specifying any combination of dimensions, positions and colours and leaving the others random, e.g. (null, null, 255, 100, 100, null, null), or (120, 87, null, 100, 100, null, null)

Solutions:

We will re-use the makeRGB function we already have.

We need to get our SVG and set the width and height, we can then use these width and height values to constrain the position and size. We do this constraining the same way we did for the colours.

Implementation:

The new function takes in up to seven parameters, x position, y position, width, height, red, green, and blue.

- First, just like our RGB string, we will give a choice to the user to either generate a completely random rectangle, or generate one with specific input choices.
- However, the random generation of x and y position as well as the width and height should all fall within the canvas - therefore, we will have to generate these scaled by the width and height of the Canvas.

```
// We first declare global width and height
// variables for our canvas.

let width = 600;
let height = 600;

// Now, we fetch the svg by its id, and set the width
// and height to these new chosen widths and heights.
let svg = document.getElementById('basesvg');
console.log(svg);
svg.setAttribute('width', width);
svg.setAttribute('height', height);
console.log(svg);

// Now we write the function to draw the rectangle
function drawRect(x, y, w, h, r, g, b) {
  // Check input and set to a random value if the input is empty.
  x = x ?? randomInt(width);
  y = y ?? randomInt(height);
  w = w ?? randomInt(width);
  h = h ?? randomInt(height);
  let fill = makeRGB(r, g, b);

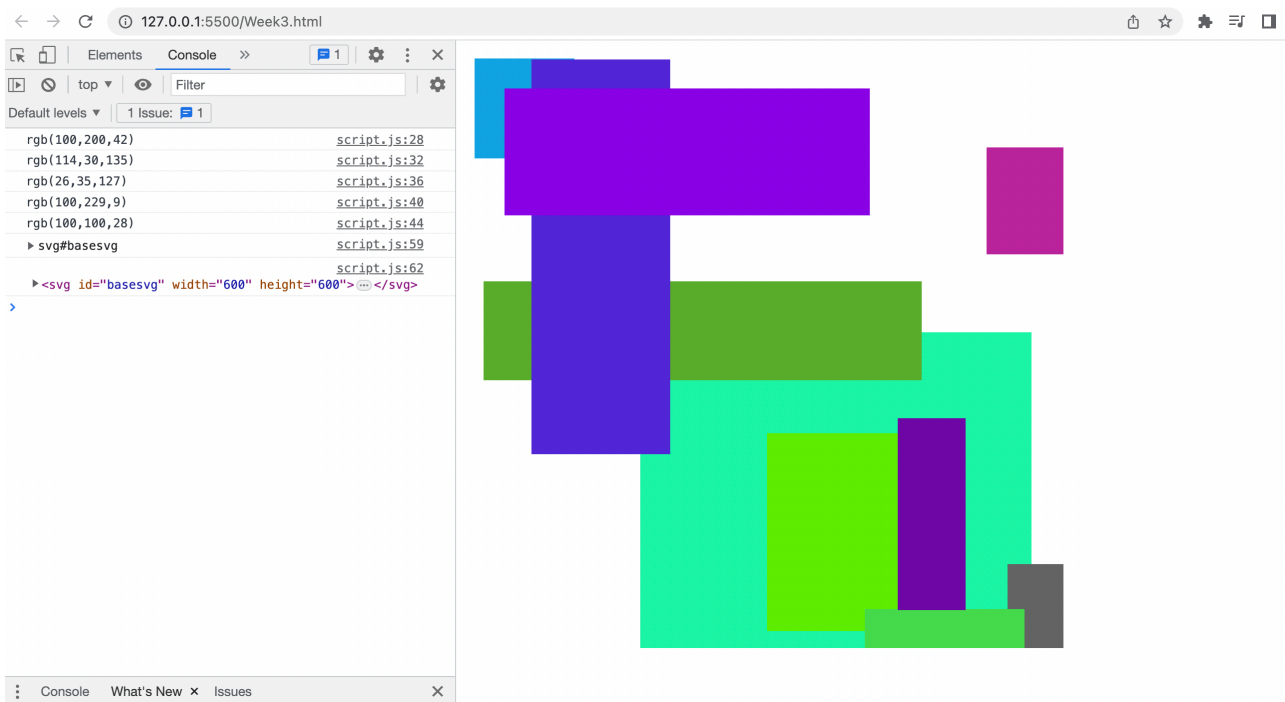
  let newRectangle = document.createElementNS("http://www.w3.org/2000/svg", "rect");

  // Apply the attributes
  newRectangle.setAttribute("x", x);
  newRectangle.setAttribute("y", y);
  newRectangle.setAttribute("width", w);
  newRectangle.setAttribute("height", h);
  newRectangle.setAttribute('fill', fill);

  svg.appendChild(newRectangle);
}

// Now we call upon this function to generate several rectangles.
drawRect();
drawRect(10, 10, 100, 100, null, null, null);
drawRect(null, null, null, null, 100, 100, 100);
drawRect();
drawRect();
drawRect();
drawRect();
drawRect();
drawRect();
drawRect();
drawRect();
```

Each time you refresh the page, you will generate a new output. This is like 101 Generative Design - using code to generate random and unexpected designs or artwork, with an extremely simple rule set:



6: Functions: Function returns with makeRect()

Now we have a function to draw a rectangle, but there is one issue, let's take a look. The last line of the function is:

```
svg.appendChild(newRectangle);
```

We know that the object called `svg` is an element we got from our HTML using dom manipulation earlier in the code. It came from this line:

```
svg = document.getElementById('basesvg');
```

When we have the line: `svg.appendChild(newRectangle);` in our function, it means that the function will only work if we have taken our SVG element from the DOM and called it `svg`. But what if we want to free that function from that dependency. What can we do?

As we saw in the lecture, functions can return things. This means when we call (run) them, they will output something, so let's use that feature to create a rectangle instead of drawing it straight away.

Objective: Change our `drawRectangle` function so that it is not locked to a specific SVG canvas, and returns a rectangle we can use however we like.

Challenges:

- Allow for position and size constraints even though we don't have the canvas.
- Make the width and height have default values that we change if we want.
- Return the rectangle from the function so we can add it to any SVG canvas we like.

Desired results: The new function will create and return a rectangle partially or completely defined by function arguments or randomly

Solutions:

- We will re-use the makeRGB function we already have.
- We will start with the drawRectangle function we have.
- We will use default values for width and height in the function definition.
- We will make our function return a rectangle object so we can add it to any SVG we like.

Implementation

We are going to name this version of the function createRect, because it doesn't draw a rectangle any more, but it creates one we can draw later.

We also need to find a similar solution for the width and height variables that are used to control the maximum size and position of the rectangle we create. In this case we are going to add default values that we can override. This means that if there are no variables called width and height in the script.js file (or wherever we use this function from), it will take the default value (in this case 500). If there are variables called width and height it will use their values, or if we want to specify width and height vales we can add those parameters to the function.

```
function createRect(x, y, w, h, r, g, b, maxWidth = 500, maxHeight = 500) {
  x = x ?? randomInt(maxWidth);
  y = y ?? randomInt(maxHeight);
  w = w ?? randomInt(maxWidth);
  h = h ?? randomInt(maxHeight);
  let fill = makeRGB(r,g,b);

  let newRectangle = document.createElementNS("http://www.w3.org/2000/svg", "rect");

  newRectangle.setAttribute("x", x);
  newRectangle.setAttribute("y", y);
  newRectangle.setAttribute("width", w);
  newRectangle.setAttribute("height", h);
  newRectangle.setAttribute('fill', fill);
  return newRectangle;
}
```

This means we have to use the function a little differently. Let's have a look at that:

```
// We first declare global width and height
// variables for our canvas.
let width = 600;
let height = 600;
// Now, we fetch the svg by its id, and set the width
// and height to these new chosen widths and heights.
let svg = document.getElementById('basesvg');
console.log(svg);
svg.setAttribute('width', width);
svg.setAttribute('height', height);
console.log(svg);
// Now we write the function to draw the rectangle
function createRect(x, y, w, h, r, g, b, maxWidth = 500, maxHeight = 500) {
  x = x ?? randomInt(maxWidth);
  y = y ?? randomInt(maxHeight);
  w = w ?? randomInt(maxWidth);
  h = h ?? randomInt(maxHeight);
  let fill = makeRGB(r, g, b);
```



```

let newRectangle = document.createElementNS("http://www.w3.org/2000/svg", "rect");

newRectangle.setAttribute("x", x);
newRectangle.setAttribute("y", y);
newRectangle.setAttribute("width", w);
newRectangle.setAttribute("height", h);
newRectangle.setAttribute('fill', fill);

return newRectangle;
}
// Here we make a new rectangle
let rectangle = createRect();
// and now we can append that rectangle to our svg
svg.appendChild(rectangle);
// We can also do all this in one step:
svg.appendChild(createRect());
// and of course our input parameters will all still work.
svg.appendChild(createRect(10, 10, 100, 100, null, null, null));
svg.appendChild(createRect(null, null, null, null, 100, 100, 100));

```

7 : Function library: Adding function definitions to a file for later

Once we have these functions defined we could keep them somewhere and copy and paste them into our code so we can use them without having to re-write them. This will work fine, but we have another option. We can add these function definitions to another file and include that file in our HTML code and use the functions from there. Let's see how it is done in VS code to create a new project with an index.html file and a script.js file and an extra file called it utils.js .

*****NOTE:

We need to be logical and careful about what we put in our utility file and what we keep in our projects script file.

The utils.js file should ONLY have things that are not unique to our project, things that are utilities we can use over and over again.

Everything else, especially calling these utility functions should be in the projects script file.

Objective: Create a utility file we can re-use in later projects so we don't have to re-write code.

Challenges:

- Make sure the utilities file can be seen by the script file so we can use it.
- Make sure we only put the definitions of useful functions in the utility file.

Desired results: An HTML file that references a script.js file and a utils.js file, where the utils file contains useful function definitions that can be used in this project or later work.

Solutions:

- We will use all the function definitions we have made today (except drawRectangle).

- We will copy these definitions into the utils.js
- We will add the utils.js to our index.html just BEFORE the script.js file.

Implementation

Set up the HTML file as follows :

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Week 3 PT 5</title>
</head>

<body>
  <svg id="basesvg" width="800" height="800"> </svg>
  <script src="utils.js">
  </script>
  <script src="script.js">
  </script>
</body>

</html>
```

utils.js

We will add the functions we made today to the utils.js file (note we are not calling any of the functions here).

```
// Generate a random integer value between 0 and the provided magnitude (limit)
function randomInt(magnitude) {
  let integer = Math.random();
  integer *= magnitude;
  integer = Math.floor(integer);

  return integer;
}

// The function takes in three parameters, red, green, and blue.
// We use a logical test to check if the user has provided
// a specific number as input, or has provided a null or
// undefined input.

// If the null condition tests true, then we generate a random number.
// Finally, we pass these into a string, which is the output of
// the function.
function makeRGB(redInputValue, greenInputValue, blueInputValue) {
  // Here, we test if there is a specific user input. If yes, then that value is assigned
  // to the output. Otherwise, a random value is generated.
  let redOutputValue = redInputValue ?? randomInt(255);
  let greenOutputValue = greenInputValue ?? randomInt(255);
  let blueOutputValue = blueInputValue ?? randomInt(255);

  // Now we define a string and pass these output values into a string.
  return `rgb(${redOutputValue}, ${greenOutputValue}, ${blueOutputValue})`;
}

// Now we write the function to create the rectangle
function createRect(x, y, w, h, r, g, b, maxWidth = 500, maxHeight = 500) {
  x = x ?? randomInt(maxWidth);
  y = y ?? randomInt(maxHeight);
```

```

w = w ?? randomInt(maxWidth);
h = h ?? randomInt(maxHeight);
let fill = makeRGB(r, g, b);

let newRectangle = document.createElementNS("http://www.w3.org/2000/svg", "rect");

newRectangle.setAttribute("x", x);
newRectangle.setAttribute("y", y);
newRectangle.setAttribute("width", w);
newRectangle.setAttribute("height", h);
newRectangle.setAttribute('fill', fill);

return newRectangle;
}

```

script.js

Now in the script.js file we can use these functions.

```

// We first declare global width and height
// variables for our canvas.
let width = 600;
let height = 600;

// Now, we fetch the svg by its id, and set the width
// and height to these new chosen widths and heights.
let svg = document.getElementById('basesvg');
console.log(svg);
svg.setAttribute('width', width);
svg.setAttribute('height', height);
svg.appendChild(createRect()); // Leave everything to default (random)
svg.appendChild(createRect(100, 100)); // specify the x and y coordinates and leave the rest to default
svg.appendChild(createRect(null, null, null, null, 100, 100, 100)); // specify the r,g,b values and leave the rest to default
svg.appendChild(createRect(null, null, null, null, null, null, null, 200, 200)); // specify the maximum width and height and leave the rest to default
svg.appendChild(createRect(100, 100, 200, 250, 100, 220, 135, 600, 600)); // specify everything

```

8: Inspiration for the week

Staying in the realm of simplicity, this week's inspiration comes from the regular (usually weekly) work of creative coder Zach Lieberman. Zach is one of the pioneers in the field of creative coding, and his contributions through developing artworks, toolkits, libraries, and educational programs have had a significant impact on a vast number of artists. Zach publishes weekly sketches on his Instagram account: <https://www.instagram.com/zach.lieberman/>  (<https://www.instagram.com/zach.lieberman/>). These sketches are quick studies that often utilise advanced or clever techniques but are always smaller in scale than his usual projects and self-contained as images.

Happy coding!

See you next week.

not remove this notice.

Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide \(https://canvas.sydney.edu.au/courses/4901/pages/zoom\)](https://canvas.sydney.edu.au/courses/4901/pages/zoom) and contact the unit coordinator.

[Privacy Statement \(https://www.sydney.edu.au/privacy-statement.html\)](https://www.sydney.edu.au/privacy-statement.html)